Medical RAG Assistant Backend Architecture

This report details the backend file architecture of the `medical-rag-assistant` GitHub repository, providing a file-by-file explanation of each component's purpose and usefulness. The backend is built using FastAPI and implements a Retrieval-Augmented Generation (RAG) system to answer medical questions based on uploaded PDF documents.

Backend File Structure Overview

The backend directory (`medical-rag-assistant/backend`) contains the following key files and subdirectories:

- `main.py` : The main entry point of the FastAPI application.
- `logger.py` : Configures logging for the application.
- `middlewares/exception_handlers.py` : Handles global exceptions.
- `modules/llm.py` : Integrates with the Large Language Model (LLM) and defines the RAG chain.
- `modules/load_vectorstore.py` : Manages document ingestion, embedding, and vector store operations.
- `modules/pdf_handlers.py` : (Potentially unused) Utility for saving uploaded PDF files.
- `modules/query_handlers.py` : Helper for processing and formatting LLM query results.
- `routes/ask_question.py` : Defines the API endpoint for asking questions.
- `routes/upload_pdfs.py` : Defines API endpoints for uploading and listing PDF documents.
- `requirements.txt` : Lists all Python dependencies.

Detailed File-wise Explanation

`main.py`

- **Purpose**: This is the core application file that initializes the FastAPI application, configures middleware, and includes all API routes.
- **Usefulness**: It serves as the central orchestrator for the backend. It sets up CORS (Cross-Origin Resource Sharing) to allow frontend applications to interact with the API, integrates a custom exception handling middleware for robust error management, and registers the API routes for uploading PDFs and asking questions. This file is essential for bringing all the backend components together and making the API functional.

logger.py

- **Purpose**: This module provides a centralized logging setup for the entire backend application.
- **Usefulness**: Effective logging is crucial for monitoring application behavior, debugging issues, and understanding the flow of data and operations. This module configures a logger that outputs messages to the console, categorizing them by level (DEBUG, INFO, ERROR, CRITICAL) and including timestamps and module names. This helps developers track the application's execution and quickly identify potential problems.

middlewares/exception_handlers.py

- **Purpose**: This middleware catches unhandled exceptions that occur during API request processing.
- **Usefulness**: By wrapping API route calls in a `try-except` block, it prevents the application from crashing due to unexpected errors. Instead, it logs the exception and returns a standardized JSON error response (HTTP 500 Internal Server Error) to the client. This improves the API's robustness and provides a consistent error handling mechanism, enhancing the user experience by preventing abrupt failures.

modules/llm.py

- **Purpose**: This module is responsible for integrating with the Large Language Model (LLM) and constructing the Retrieval-Augmented Generation (RAG) chain.
- **Usefulness**: It defines the `get_llm_chain` function, which initializes a `ChatGroq` LLM (using the `llama-3.3-70b-versatile` model) and a `PromptTemplate`. The prompt guides the LLM, named "MediBot", an AI-powered assistant, to provide accurate and helpful responses based *only* on the provided context. It also sets up the `RetrievalQA` chain, which combines the LLM with a retriever to fetch relevant documents before generating answers. This file is central to the RAG functionality, ensuring that the LLM's responses are contextually relevant and avoid factual inaccuracies.

modules/load_vectorstore.py

- **Purpose**: This module handles the ingestion of PDF documents, their processing, embedding, and storage in a vector database (Pinecone).
- **Usefulness**: This is a critical component for the RAG system. It performs the following key steps:
    1. **Environment Setup**: Loads API keys and configurations for Pinecone and Google Gemini embeddings.

2. **Vector Index Management**: Initializes the Pinecone client and creates a `medicalindex` if it doesn't exist, defining its dimension and similarity metric.
3. **Document Parsing**: Uses `PyPDFLoader` to extract text from uploaded PDF files.
4. **Text Chunking**: Breaks down the extracted text into smaller, manageable chunks using `RecursiveCharacterTextSplitter` to optimize retrieval.
5. **Embedding Generation**: Converts these text chunks into numerical vector embeddings using `GoogleGenerativeAIEmbeddings`.
6. **Vector Upsertion**: Stores these embeddings, along with their metadata, into the Pinecone vector index. This process transforms raw documents into a searchable format, enabling efficient retrieval of relevant information during question-answering.

routes/upload_pdfs.py

- **Purpose**: This file defines the API endpoints related to uploading PDF documents and listing previously uploaded documents.
- **Usefulness**: It provides the interface for users to add new medical documents to the system. The `/upload_pdfs/` endpoint receives PDF files, delegates their processing to `modules.load_vectorstore.load_vectorstore`, and confirms the upload. The `/list_documents/` endpoint allows users to see which documents are currently available in the system. This file is essential for populating and managing the knowledge base of the RAG assistant.

routes/ask_question.py

- **Purpose**: This file defines the API endpoint for users to submit questions to the RAG assistant.
- **Usefulness**: This is the primary interaction point for users seeking answers. The `/ask/` endpoint receives a user's question, then orchestrates the following:
  1. **Question Embedding**: Converts the user's question into a vector embedding.
  2. **Vector Search**: Queries the Pinecone vector database to find the most relevant document chunks based on the question's embedding.
  3. **Retrieval and LLM Integration**: Uses the retrieved chunks as context for the LLM (via `get_llm_chain` from `modules/llm.py`) to generate a coherent and informed answer.
  4. **Response Formatting**: Formats the LLM's response, including the answer and the sources of information. This file ties together the retrieval and generation components to deliver the core functionality of the medical RAG assistant.

modules/query_handlers.py

- **Purpose**: This is a helper module containing a function to process and format the output from the LangChain LLM chain.
- **Usefulness**: The `query_chain` function simplifies the extraction of the answer and source documents from the LLM's raw output. It ensures that the final response presented to the user is clean and structured, making it easier to consume. It also handles logging of the input and potential exceptions during the query process.

modules/pdf_handlers.py

- **Purpose**: This module contains a utility function for saving uploaded files locally.
- **Usefulness**: While it appears to be present, the current implementation of `routes/upload_pdfs.py` directly calls `modules.load_vectorstore.load_vectorstore`, which handles file saving internally. Therefore, this specific `pdf_handlers.py` module might be an older or unused component in the current backend flow. If it were actively used, its purpose would be to provide a reusable function for securely storing uploaded PDF files on the server's filesystem.

requirements.txt

- **Purpose**: This file lists all the Python dependencies required to run the backend application.
- **Usefulness**: It ensures that the development and deployment environments have all the necessary libraries installed, maintaining consistency and reproducibility. It categorizes dependencies into groups like FastAPI web stack, LangChain components, Pinecone client, and embedding models, providing a clear overview of the technology stack used.

Conclusion

The backend of the `medical-rag-assistant` is a well-structured FastAPI application that effectively implements a Retrieval-Augmented Generation (RAG) system. It separates concerns into distinct modules for web routing, logging, exception handling, LLM integration, and vector store management. This modular design enhances maintainability, scalability, and clarity, making it a robust foundation for a medical question-answering system.